

Draft TASK 3 :

Problem Description, Cost Function, Sources
Methodology and settings
Result
Work Discussion

Global objectives: Compare **Particle Swarm Optimisation** and **Simulated Annealing**

Table des matières

1.	Method 2 : Solid Annealing.....	13
1.1.	Problem description:	13
1.2.	Cost function:	13
1.3.	Sources:	13
1.4.	Methodology:	13
1.5.	Settings:	13
2.	Problem 3 : Sensor Calibration	14
2.1.	Problem description:	14
2.2.	Cost function:	14
2.3.	Sources:	14
2.4.	Methodology:	14
2.5.	Settings:	14
2.6.	Result :	15
2.7.	Work discussion :	18
3.	Problem 4: Gear Train Design.....	18
3.1.	Problem description	18
3.2.	Cost function	19

3.3.	Sources	19
3.4.	Methodology	19
3.5.	Settings	20
3.6.	Results	20
3.6.1.	Results analysis	20
3.6.2.	Explanations	22
3.6.3.	Conclusion	23

Table des figures

Figure 1: Two-bar plane truss [4]	4
Figure 2 : PSO Results	20
Figure 3 : PSO Convergence curve	21
Figure 4 : SA Results	21
Figure 5 : SA Convergence Curve	22

2.1 Method 1: Particle Swarm Optimisation (PSO)

The Particle Swarm Optimisation (PSO) is a stochastic optimisation technique inspired by the social behaviour of birds. A population of candidate solutions moves through the domain. Through several time steps, each particle adjusts its position based on its personal best and on the global best. This allows the algorithm to converge to a solution that matches a criterion. [1]

2.1.1 Multi-objective algorithm

2.1.1.1 Description

The multi-objective PSO finds Pareto-optimal solutions using a selection based on dominance and diversity [2]. It presents advantages such as an ease of implementation, few

parameters to tune, and a good application for nonlinear problems; but also, limitations such as a possible too early convergence, a sensitivity to parameters settings, and no guarantee of global optimum [1].

The MOPSO-CD (Multi-Objective Particle Swarm Optimisation with Crowding Distance) is an advanced an extension of the PSO [3]. This particular algorithm was chosen from a list of other solution on the base of specific criteria (Appendix 8.1). Its key features are:

- Crowding distance-based leader selection,
- External archive management,
- Tournament selection,
- Enhanced exploration.

2.1.1.2 Performance metrics

Metrics are used to measure the performance of the algorithm. We have selected some of the most common criteria:

- The best value per objective which tracks the minimum value achieved for each objective function at every generation,
- The Pareto front size evolution which tracks how many solutions are on the Pareto front at each generation,
- The normalized convergence which compares both objectives normalized to [0, 1].

2.1.2 Single-objective algorithm

Martin

3 Problem 1: Two-Bar Plane Truss Design

3.1 Problem description [4]

The Two-Bar Truss problem is composed by two diagonal bars (of aluminium here) connected on a node and are fixed at two upper point (Figure 1). On the central node we applied a vertical strength. This force represents the external load that the structure must support. We look for the vertical displacement of the node.

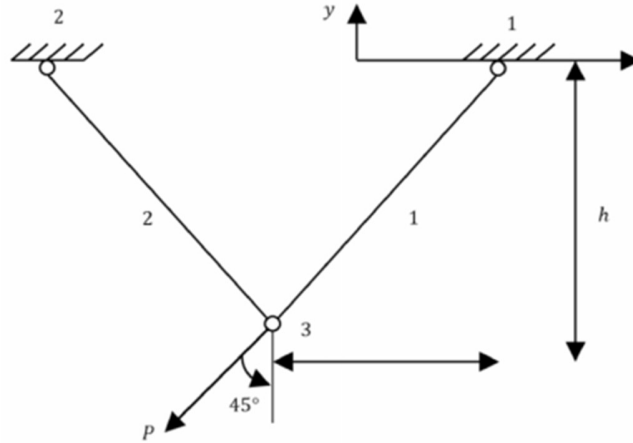


Figure 1: Two-bar plane truss [4]

There are two design variables which are the position of the joints 1 and 2. They are defined by the two equations: f_1 the structural weight truss, and f_2 the displacement of joint 3.

$$f_1(x) = 2\rho h x_2 \sqrt{1+x_1^2}$$

$$f_2(x) = \frac{Ph(1+x_1^2)^{1.5} \sqrt{1+x_1^4}}{2\sqrt{2}Ex_1^2x_2}$$

$$f_1(x) = 2\rho h x_2 \sqrt{1+x_1^2}$$

$$f_2(x) = \frac{Ph(1+x_1^2)^{1.5} \sqrt{1+x_1^4}}{2\sqrt{2}Ex_1^2x_2}$$

These functions are constrained by two stress constraints functions g_1 (tension) and g_2 (compression).

$$g_1(x) = \frac{P(1+x_1) \sqrt{1+x_1^2}}{2\sqrt{2}x_1x_2} - \sigma_0 \leq 0$$

$$g_2(x) = \frac{P(-x_1+1) \sqrt{1+x_1^2}}{2\sqrt{2}x_1x_2} - \sigma_0 \leq 0$$

The variables x_1 and x_2 are calculated from the following formulas:

$$x_1 = \frac{x}{h} \text{ and } x_2 = \frac{A}{A_{\min}}$$

$$0.1 \leq x_1 \leq 2.25 \text{ and } 0.5 \leq x_2 \leq 2.5$$

If the x_1 variable is connected to the physical sense of the truss geometry, the position boundaries are:

$$x_{\min} = 0.1 \text{ at } h = 0.254 \text{ m and } x_{\max} = 2.25 \text{ at } h = 5.72 \text{ m}$$

The parameters are converted in the International System units.

$$\rho = 0.283 \text{ lb/in}^3 = 7833 \text{ kg/m}^3$$

$$h = 100 \text{ in} = 2.54 \text{ m}$$

$$P = 10^4 \text{ lb} = 44482.2 \text{ N}$$

$$E = 3 \times 10^7 \text{ lb/in}^2 = 2.07 \times 10^{11} \text{ Pa}$$

$$\sigma_0 = 2 \times 10^4 \text{ lb/in}^2 = 1.38 \times 10^8 \text{ Pa}$$

$$A_{\min} = 1 \text{ in}^2 = 6.5416 \times 10^{-4} \text{ m}^2$$

3.2 Methodology

3.2.1 General explanation

The problem is defined with a class which contained four functions: two for the objectives and two for the constraints.

Add something? → explanation of the class structure

3.2.2 Penalty function

Pymoo library already contains constraint handling with constraint violation measures during the selection process. For the multi-objective problem, it uses the Pareto ranking. To study how the penalty strength affects convergence and feasibility, explicit penalty terms will be added [2].

$$F_{\text{penalised}} = f(x) + k \sum \left(\max \left(0, g_i(x) \right) \right)^2$$

With: k the penalty factor

Considering this equation, a penalty will be applied when the maximum between 0 and g_i is g_i so the constraint is violated. This is realised with the following loop where the penalty factor is fixed to 10 billion. If the penalty factor is large the algorithm avoids infeasible solutions quickly. While, if the penalty factor is small the algorithm may explore infeasible regions longer.

```
for i in range(n_solutions):
    try:
        objectives, constraints = self.truss.evaluate(x[i, :])
        F[i, :] = objectives
        G[i, :] = constraints
    except ValueError:
        # Penalty for invalid solutions
        F[i, :] = [1e10, 1e10]
        G[i, :] = [1e10, 1e10]
```

3.3 Results

3.3.1 PSO simulation

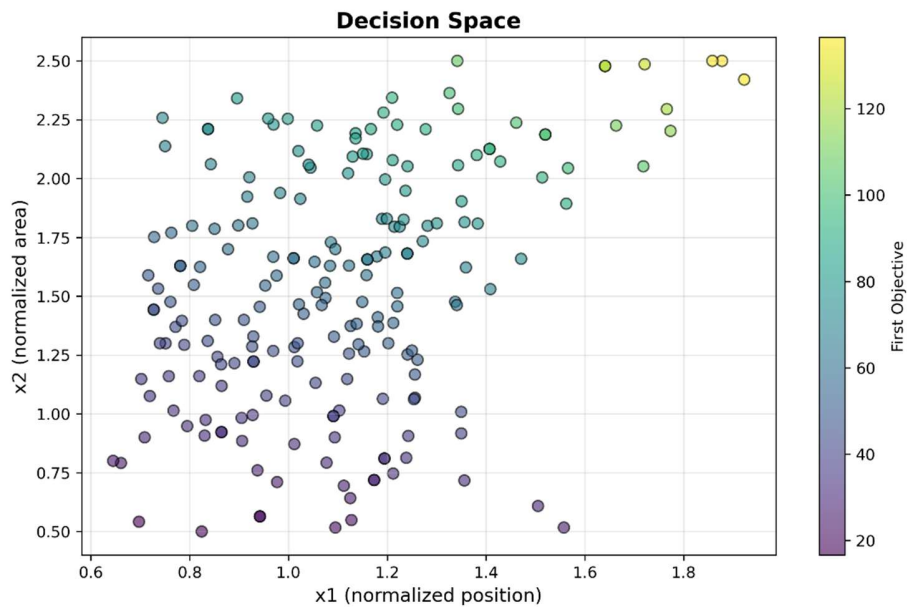
3.3.1.1 Best solution

To obtain the best solution, the algorithm minimises the weight function and then the displacement function. Then it finds the best compromise to satisfy the objective functions following the constraint function.

Table 1: Summary of key solutions

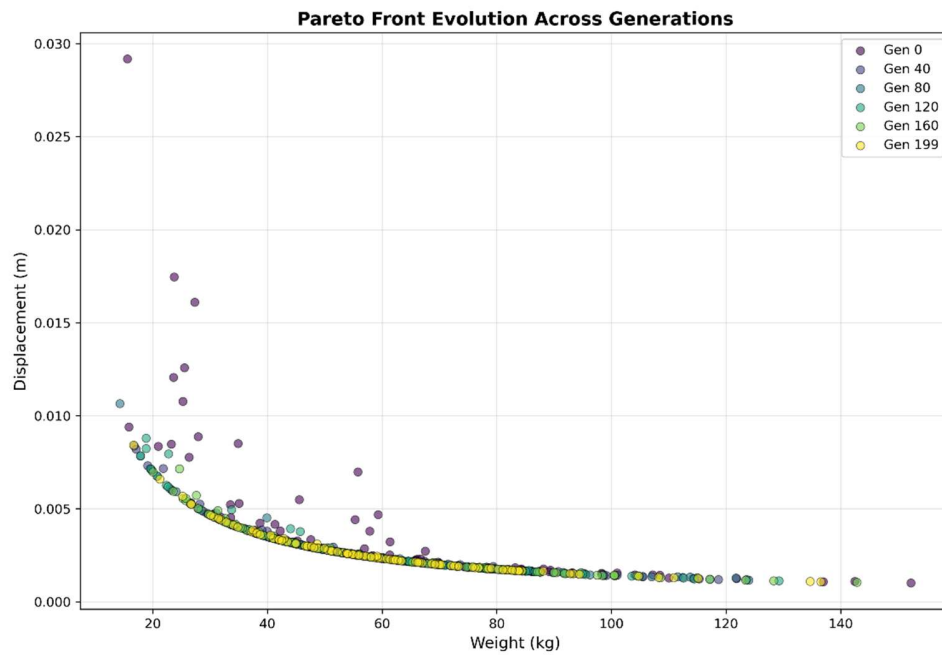
Solution type	x1 [normalised]	x2 [normalised]	Weight [kg]	Displacement [m]
Min weight	0.8245	0.5	16.6361	0.0084
Min displacement	1.8777	2.5	136.5345	0.0011
Compromise	1.2560	1.1667	48.0982	0.0029

All the possible values of the design variables are shown in the decision space diagram: each point represents a candidate solution. The colour show a gradient: dark colours for the lower objective values which are better for minimisation, and lighter colours for the higher objective values which are worse for minimisation.

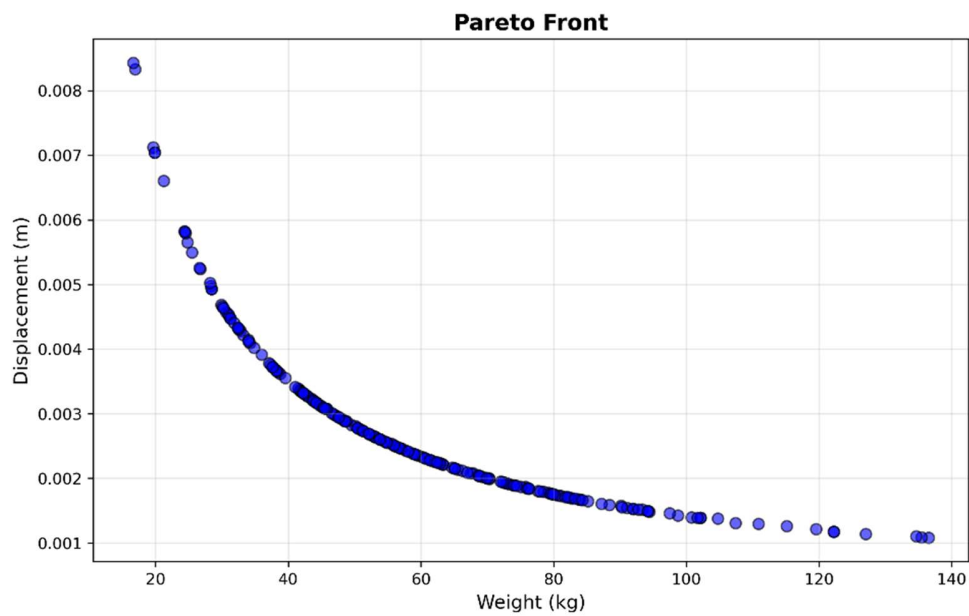


3.3.1.2 Metrics

The Pareto front evolution across generations diagram shows that the first generations are far away from the Pareto front but through each time step the solutions converge. This is possible thanks to the penalty function which eliminates poor solutions. The visualisation helps identify regions of the decision space associated with optimal solution.



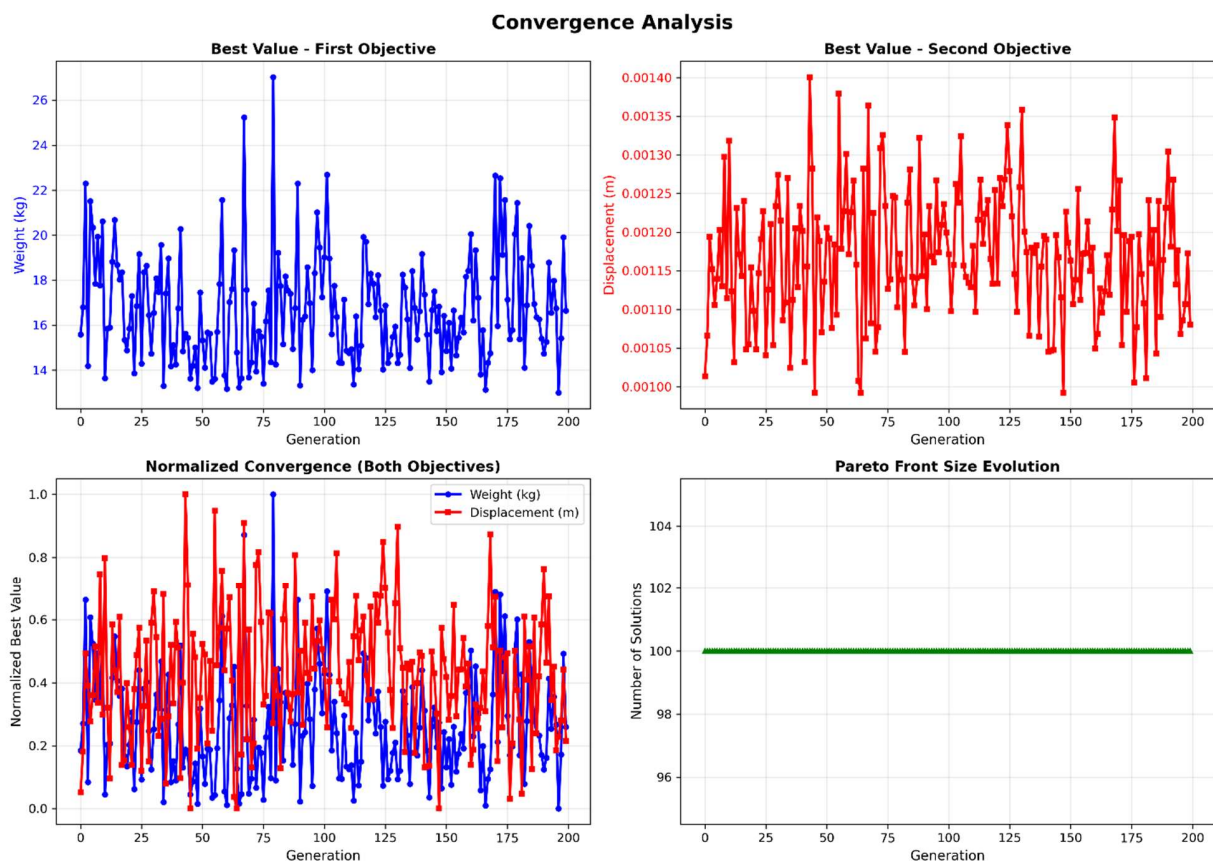
The generation 199, or 200 time step, generates the Pareto front kept. The best solution is extracted from it.



The convergence analyses the efficiency of the algorithm. The following diagrams describes the evolution of the best values at each generation.

- Best value – First objective:
 - The weight is minimised over time with an improvement of the solution.
 - The curve decreases over the generation, so the minimum is stable.
- Best value – Second objective:
 - The algorithm finds solutions with smaller solutions through the generations.
 - The curve converges around a optimal displacement.
- Normalised convergence:
 - The objectives are normalised for comparison.
 - The algorithm is improving both objectives simultaneously since the curves decreases.

The Pareto front size evolution diagram shows the number of non-dominant solutions at each generation. The constant size of it means diversity is maintained throughout optimisation. This is a good indicator to tell that the algorithm didn't lose diversity and didn't converge prematurely.



3.3.2 SA simulation

Pierre

3.4 Discussion

Comparison between PSO and SA simulation → Results and metrics

Algorithm	x1 [normalised]	x2 [normalised]	Weight [kg]	Displacement [m]
PSO	1.2560	1.1667	48.0982	0.0029
SA				

7. References

1. Riaz A. Computational Optimisation Design Lecture 2 [Internet]. Lecture presented at: Computational Optimisation Design. 2025 [cited 2025 Oct 21]. Available from: https://canvas.cranfield.ac.uk/courses/37635/pages/day-2?module_item_id=737718
2. Riaz A. Computational Optimisation Design Lecture 3 [Internet]. Lecture presented at: Computational Optimisation Design. 2025 [cited 2025 Oct 22]. Available from: https://canvas.cranfield.ac.uk/courses/37635/pages/day-3?module_item_id=737719
3. Blank J. MOPSO-CD: Multi-Objective Particle Swarm Optimization with Crowding Distance [Internet]. Pymoo library. 2020 [cited 2025 Dec 9]. Available from: https://pymoo.org/algorithms/moo/mopso_cd.html
4. Chiandussi G, Codegone M, Ferrero S, Varesio FE. Comparison of multi-objective optimization methodologies for engineering applications. Computers and Mathematics with Applications [Internet]. 2012 Mar [cited 2025 Dec 4];63(5). Available from: <https://www.sciencedirect.com/science/article/pii/S0898122111010406>

8. Appendix

8.1 Multi-objective Particle Swarm Optimisation choice

To implement the Particle Swarm optimisation, a list of criteria with weight is established.

Table 2: Criteria for selecting the solution for coding Particle Swarm Optimisation

Criteria	Description	Weight
Objective handling capability	Ability to handle both single-objective and multi-objective optimisation problems	6
Method sophistication	Use of advanced optimisation methods rather than basic scalarisation approaches	4
Academic research fit	Suitability and alignment with academic research requirements	1
Algorithm performance	Quality and effectiveness of the optimisation algorithms	3
Swarm-based approach	Use of particle swarm optimisation methodology	5
Documentation	Good documentation of the solution	2

GitHub repositories purpose solutions but it doesn't respond to the need. The searches are therefore concentrated on libraries.

Table 3: Ranking of possible solutions

Solution		Objective handling capability	Method sophistication	Academic research fit	Algorithm performance	Swarm-based approach	Documentation
Pymoo library	COMPSO method	6	4	1	3	5	2
	MOPSO-CD method	6	4	1	2	5	2
	NSGA-III	6	4	1	3	0	2
	NSGA-II	6	4	1	3	0	2

	R-NSGA-II	6	4	1	3	0	2
Platypus library	NSGA-II	6	4	0	3	0	1
	NSGA-II	6	4	0	3	0	1
	MOEA/D	6	4	0	3	0	1
Custom MOPSO		6	4	0	2	5	0
PySwarms library	Weighted sum	3	0	0	1	5	2
	Epsilon-constraint	3	2	0	1	5	2

According to Table 2, the best library is Pymoo and the best algorithm solution is COMPSO method.

8.2 Python code

8.2.1 ...

1. Method 2 : Solid Annealing

1.1. Problem description:

The Simulated Annealing is an optimization method inspired by the annealing process in metallurgy. The algorithm explores the search space and accepts neighbouring solutions. In this algorithm, ameliorations are always accepted, and degradations are accepted with a probability. The probability decreases step after step (decreasing temperature). At the beginning, this acceptance mechanism helps to escape local minima. Afterwards, at the end of the process, the algorithm converges to a global minimum.

1.2. Cost function:

On this method we use this function to decide if a worst solution can be accepted:

$$P(\text{acceptance}) = e^{\{-\frac{E_{\text{new}} - E_{\text{actual}}}{T}\}}$$

E_{actual} : Actual energy

E_{new} : The new worst energy

$P(\text{acceptance})$: the probability to accept E_{new}

T : temperature parameter

Every iteration the temperature T decrease of cooling_rate:

$$T *= \text{cooling_rate}$$

1.3. Sources:

- <https://www.youtube.com/watch?v=eBmU1ONJ-os>

1.4. Methodology:

1.5. Settings:

It is important to consider:

- initial_temp : the temperature at the beginning
- cooling_rate : The speed of the freezing
- min_temp : The temperature where we stop the process

2. Problem 3 : Sensor Calibration

2.1. Problem description:

The objective is to determine the Steinhart-Hart coefficients (A, B, C) that model the relationship between Resistance (R) and Temperature (T) for a 10K thermistor.

Model Equation: $T_{\{model\}} = \frac{1}{A+B*\ln(R)+C*(\ln(R))^3}$ Note: T is in Kelvin, R in ohms

2.2. Cost function:

The optimisation goal is to minimise the Sum of Squared Errors (SSE) between the measured temperature and the model-predicted temperature:

$$SSE = \sum (T_{\{measured,i\}} - T_{\{model,i\}})^2$$

2.3. Sources:

https://www.bapihvac.com/wp-content/uploads/2010/11/Thermistor_10K-2.pdf

<https://www.sciencedirect.com/science/article/abs/pii/S0011747168900570>

2.4. Methodology:

Simulated Annealing :

- Description: A probabilistic technique for approximating the global optimum of a given function.

Particle Swarm Optimisation (PSO) :

- Description: A computational method that optimises a problem by iteratively trying to improve a candidate solution based on a given measure of quality. It solves a problem by maintaining a population of candidate solutions, here dubbed particles, and moving these particles around in the search space according to a simple mathematical formula that depends on the particles' positions and velocities.

2.5. Settings:

- Settings SA:
 - Initial Temperature: 50.0

- Cooling Rate: 0.995
- Minimum Temperature: 1e-6

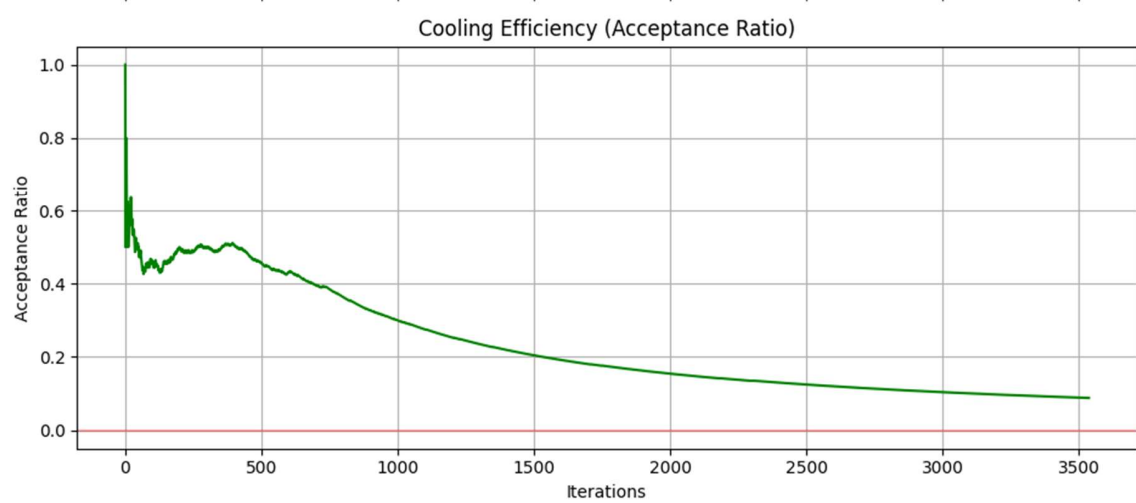
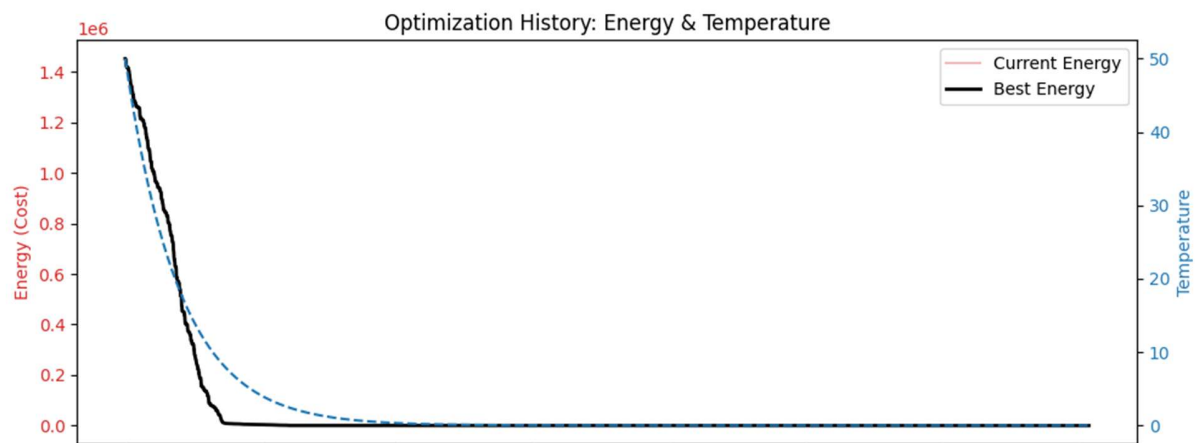
Setting PSO: Implementation: Utilises the pymoo library (PSO algorithm).

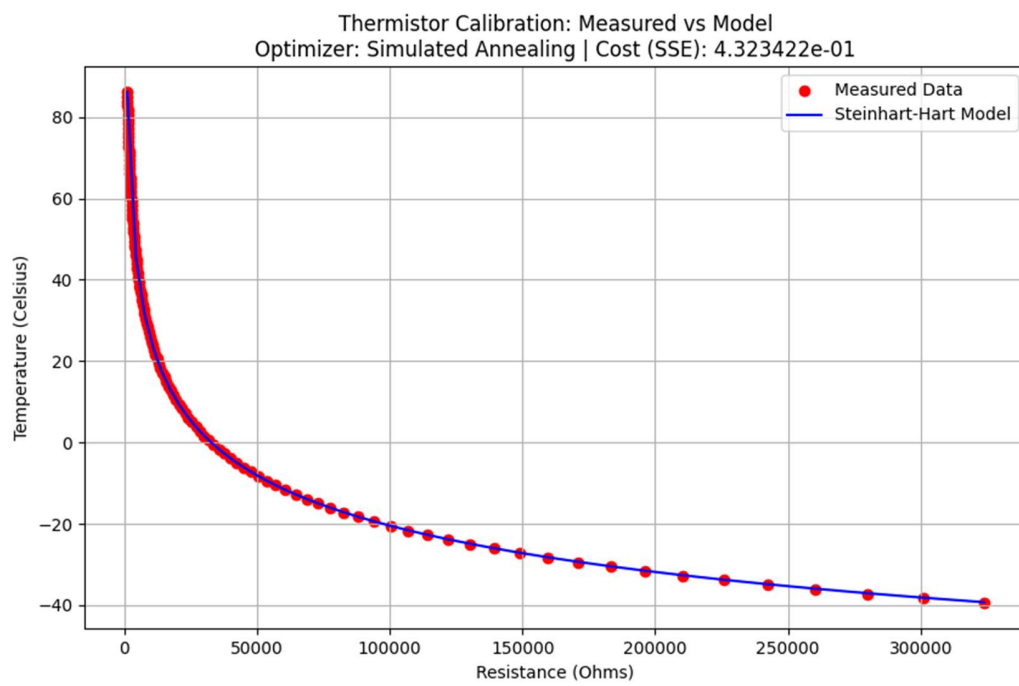
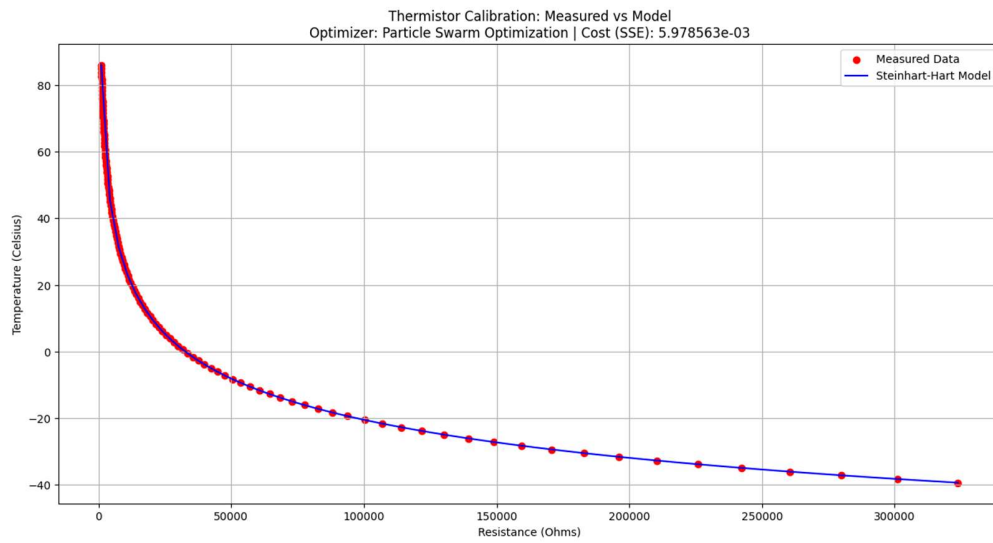
- pop_size: Population size (controls diversity).
- w: Inertia weight (balance between exploration and exploitation).
- c1: Cognitive parameter (individual experience).
- c2: Social parameter (swarm experience).
- max_velocity_rate: Limits the maximum velocity of particles.

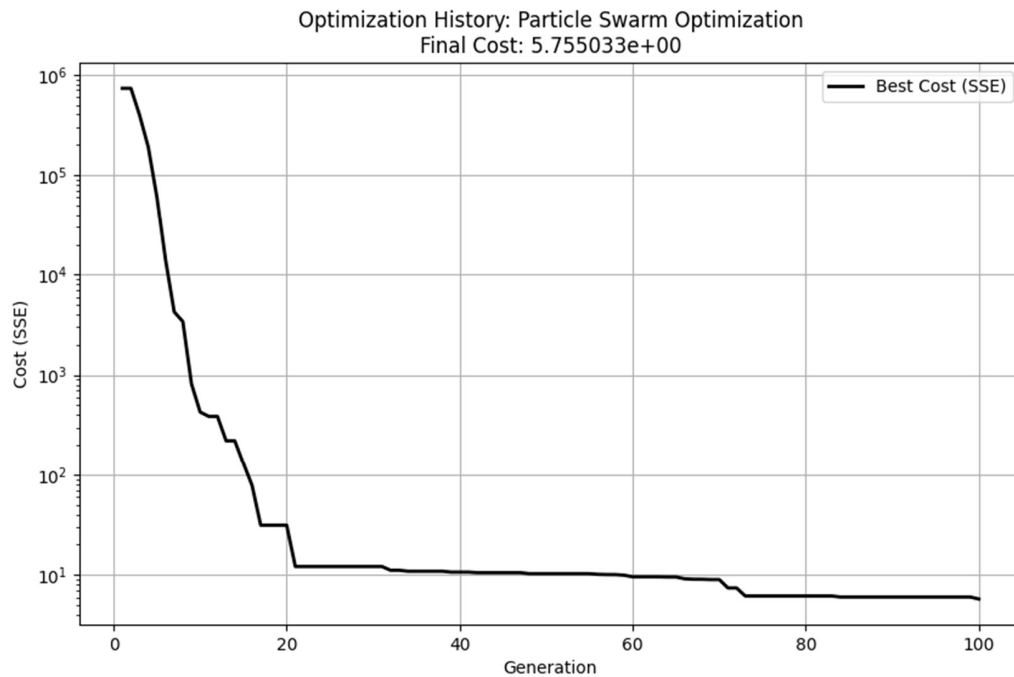
2.6. Result :

The optimal Steinhart-Hart coefficients found for the provided dataset are:

Method	A	B	C	Final SSE
Simulated Annealing	1.114203e-03	2.366744e-04	7.761151e-08	4.323422e-01
PSO	1.127363e-03	2.344235e-04	8.655284e-08	5.978563e-03







2.7. Work discussion :

3. Problem 4: Gear Train Design

3.1. Problem description

The objective of this exercise is to choose a gear train design that achieves a specific transmission ratio.

We would define the gear train with four gears with the numbers of teeth N_1, N_2, N_3, N_4

The target transmission ratio is: $R = \frac{N_2 \cdot N_4}{N_1 \cdot N_3}$

Here, the target transmission ratio will be randomly chosen: $R_{target} = \frac{1}{6.931} \approx 0.1142793$

And each gear must have an integer number of teeth between 12 and 60.

The problem is difficult because the search space is discrete and nonlinear, meaning classical optimization cannot be used directly.

Therefore, two heuristic optimization algorithms will be tested:

- Particle Swarm Optimization (PSO)
- Simulated Annealing (SA)

The goal is to compare their performance and their behavior on this discrete optimization problem.

3.2. Cost function

The cost function measures the error between the achieved ratio and the target ratio:

$$f(N_1, N_2, N_3, N_4) = \left| \frac{N_1 \times N_3}{N_2 \times N_4} - R_{t \text{ arg}} \right|$$

A good solution is one where this error approaches zero.

3.3. Sources

Course material on PSO and Simulated Annealing

3.4. Methodology

Particle Swarm Optimization:

It simulates a swarm of particles exploring the search space. Each particle represents a potential gear combination $[N_1, N_2, N_3, N_4]$.

Particles move according to:

- Their own best-found solution
- The best solution found by the swarm

Positions are rounded to integers to satisfy the gear teeth constraint as we can't make gears with an odd number of teeth.

We store the best error at each iteration to generate a convergence curve like we did for the SA.

Simulated Annealing:

It explores the space by randomly perturbing the current solution. It accepts worse solutions with a probability depending on analogy to a temperature parameter:

- With high temperature goes a wide exploration
- Low temperature goes a convergence around a good solution

The algorithm stops when the temperature becomes very low. We record both the current energy and the best energy to produce a convergence curve.

3.5. Settings

PSO

- Population size: 40
- Iterations: 300
- $w = 0.7, C_1 = 1.5, C_2 = 1.5$
- Integer rounding of particle positions
- Recorded global best error per iteration (for convergence curve)

SA

- Initial temperature: 1000
- Cooling rate: 0.90
- Minimum temperature: 0.1
- Random seed: 42
- Recorded: Current energy / Best energy (for convergence curve)

3.6. Results

3.6.1. Results analysis

The optimal solution found by the Particle Swarm Optimization is:

```
--- PSO Finished in 300 iterations ---  
Final PSO Energy: 0.0000  
=== PSO RESULT ===  
Best gear combination: [17, 60, 28, 55]  
Achieved ratio: 0.14424242424242426  
Target ratio: 0.14427932477276006  
Absolute error: 3.690053033580942e-05
```

Figure 2 : PSO Results

PSO finds an extremely accurate gear ratio, with an error on the order of 10^{-5} , meaning it almost exactly matches the target ratio.

PSO finds an extremely accurate gear ratio, with an error on the order of 10^{-5} , meaning it almost exactly matches the target ratio.

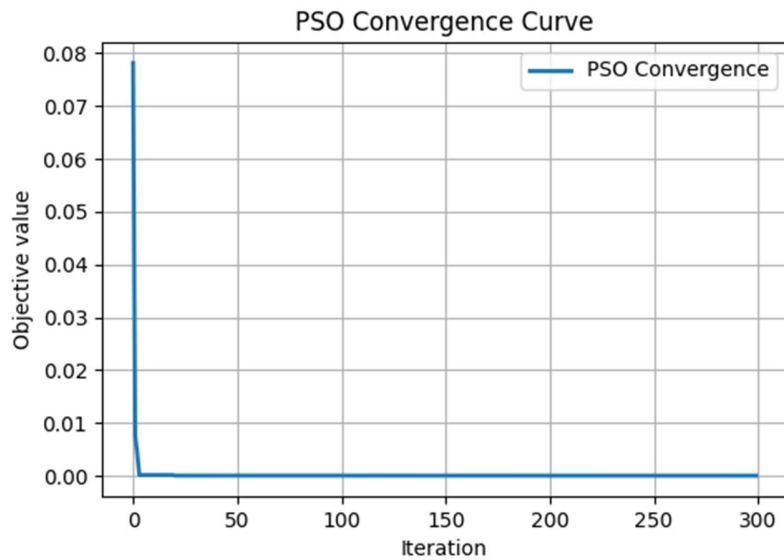


Figure 3 : PSO Convergence curve

The graph shows that PSO converges extremely quickly within ~10 iterations and then stabilizes at a very small error value.

And the result found by the Solid Annealing is:

```
Random Seed set to: 42
--- Starting Optimization ---
--- Finished in 88 iterations ---
Final Energy: 2.9650
Global Acceptance Ratio: 85.23%
=== SIMULATED ANNEALING RESULT ===
Best gear combination: [37, 21, 30, 17]
Achieved ratio: 3.1092436974789917
Target ratio: 0.14427932477276006
Absolute error: 2.9649643727062314
```

Figure 4 : SA Results

SA clearly fails on this problem. The final ratio is far from the target, with a very large error.

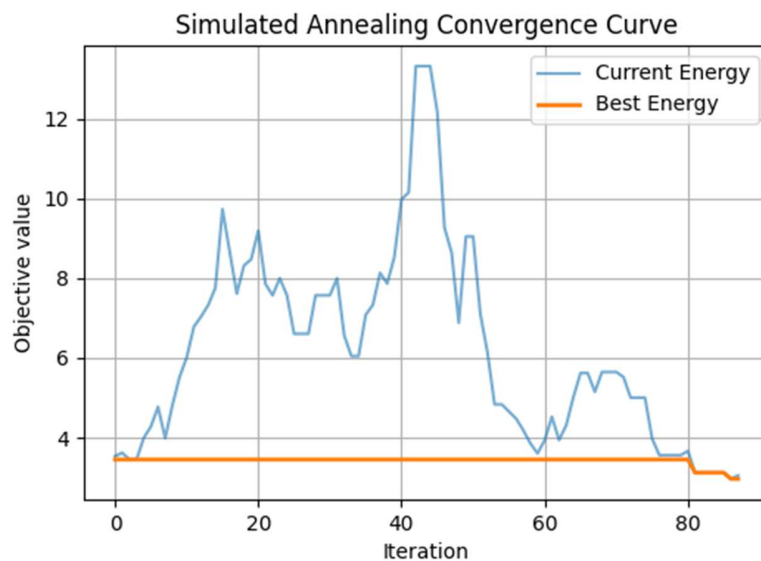


Figure 5 : SA Convergence Curve

The plot shows:

- Highly unstable exploration, current energy jumps up and down
- Best energy stays flat because the algorithm never discovers a good solution
- Cooling happens too fast to explore thoroughly

3.6.2. Explanations

Why PSO succeeds

- PSO is particularly effective in structured continuous search spaces, even with integer rounding.
- The particles rapidly converge to the correct ratio because the cost function is smooth.
- Global and personal best mechanisms guide the search efficiently.

Why SA fails

- The gear train problem is very sensitive: small changes in N_i produce large jumps in the ratio.
- SA explores randomly and relies on thermal acceptance, which:
 - Moves too chaotically
 - Does not exploit promising regions enough
 - Requires much slower cooling to perform well

- SA found a physically valid gear set but not even close to the desired ratio.

3.6.3. Conclusion

PSO is clearly the superior method for this task. It finds a nearly perfect solution extremely fast, while SA remains stuck far away from any optimal region.

For a future study, SA could be improved by:

- Reducing the cooling rate from 0.99 to 0.90 for example
- Using a smarter neighbour generator
- Introducing integer-specific heuristics

But in its current form, SA is not suitable for this optimisation problem.